

The suggested system incorporates consumer purchasing behaviour as a feature. Word2vec is used to create a vector representation of the products. It provides low-dimensional (50–500) and dense (not sparse; most values are non-zero) word embedding formats. As a result, if a user is browsing a product online, we can immediately suggest related items based on the product's vector similarity score.

After gathering the necessary data and performing a train-test split on it, an attempt was made to create word and sentence equivalents comparable to those of a standard word2Vec model, which was then fed to the model for training.

We will try and explain the implementation by dividing it in three parts:

- Data pre-processing
- Exploratory data analysis
- Model building

Data pre-processing

We will first take a quick look at the data set in this segment that we are going to evaluate, to see how we can make it easier and more useful for further research.

Let's get started by importing the necessary libraries and the data set we'll be working with. Because the 'InvoiceDate' attribute is in an unsatisfactory format, we fix it by using this date variable to format our desired type. We also have some wrong spellings and excessive white spaces in our strings that we would like to prevent, and we would like to keep our strings in upper case for the next steps.

We will install *openpyxl* using the command given below:

```
!pip install openpyxl

Collecting openpyxl
  Downloading openpyxl-3.0.9-py2.py3-none-any.whl (242 kB)
    |████████████████████████████████████████| 242 kB 4.2 MB/s
Collecting et_xmlfile
  Downloading et_xmlfile-1.1.0-py3-none-any.whl (4.7 kB)
Installing collected packages: et-xmlfile, openpyxl
Successfully installed et-xmlfile-1.1.0 openpyxl-3.0.9
```

WARNING: Running pip as the 'root' user can result in broken permissions and

conflicting behaviour with the system package manager. It is recommended to use a virtual environment instead (<https://pip.pypa.io/warnings/venv>).

Next, we will import the necessary libraries and the data set:

```
import numpy as np, pandas as pd, re,
    scipy as sp, scipy.stats

#Importing Dataset
pd.options.mode.chained_assignment = None
datasetURL = 'https://query.data.world/s/e42xdsj4d3a6evs7t3z6kjoj4eds1'
df1 = pd.read_excel(datasetURL)
original_df=df1.copy()
original_df.head()

df1.size
df1.isnull().sum().sort_
values(ascending=False)
df1=df1.dropna(axis=0)
df1.isnull().sum()
#Formatting Date/Time
df1['InvoiceDate'] = pd.to_
datetime(df1['InvoiceDate'], format =
'%m/%d/%Y %H:%M')
```

Figure 1 shows the output of the above command.

InvoiceNo	StockCode	Description	Quantity	InvoiceDate	UnitPrice	CustomerID	Country
0	536365	85123A WHITE HANGING HEART T-LIGHT HOLDER	6	2010-12-01 08:26:00	2.55	17850.0	United Kingdom
1	536365	71053 WHITE METAL LANTERN	6	2010-12-01 08:26:00	3.39	17850.0	United Kingdom
2	536365	84406B CREAM CUPID HEARTS COAT HANGER	8	2010-12-01 08:26:00	2.75	17850.0	United Kingdom
3	536365	84029G KNITTED UNION FLAG HOT WATER BOTTLE	6	2010-12-01 08:26:00	3.39	17850.0	United Kingdom
4	536365	84029E RED WOOLLY HOTTIE WHITE HEART.	6	2010-12-01 08:26:00	3.39	17850.0	United Kingdom

Figure 1: Imported data set

InvoiceNo	StockCode	Description	Quantity	InvoiceDate	UnitPrice	CustomerID	Country
0	536365	85123A WHITE HANGING HEART T-LIGHT HOLDER	6	2010-12-01 08:26:00	2.55	17850.0	UNITED KINGDOM
1	536365	NaN WHITE METAL LANTERN	6	2010-12-01 08:26:00	3.39	17850.0	UNITED KINGDOM
2	536365	84406B CREAM CUPID HEARTS COAT HANGER	8	2010-12-01 08:26:00	2.75	17850.0	UNITED KINGDOM
3	536365	84029G KNITTED UNION FLAG HOT WATER BOTTLE	6	2010-12-01 08:26:00	3.39	17850.0	UNITED KINGDOM
4	536365	84029E RED WOOLLY HOTTIE WHITE HEART	6	2010-12-01 08:26:00	3.39	17850.0	UNITED KINGDOM

Figure 2: Output after string formatting